

Exercise 1 – Hello MPI

Objective

To execute a basic MPI program using multiple processes and observe the rank and processor name of each process.

Program

The `HelloMPI.c` program initializes MPI using `MPI_Init()`, obtains the rank of each process using `MPI_Comm_rank()`, and obtains the processor name using `MPI_Get_processor_name()`.

Rank 0 displays a **"Hello World"** message, while the other processes display a normal process message.

Compilation

```
mpicxx HelloMPI.c -o HelloMPI
```

Execution

The program was executed using 1, 2, 4, and 8 MPI processes.

```
mpirun -np 1 ./HelloMPI
mpirun -np 2 ./HelloMPI
mpirun -np 4 ./HelloMPI
mpirun --oversubscribe -np 8 ./HelloMPI
```

Results

Number of Processes Execution Time

1	0.413 s
2	0.574 s
4	0.836 s
8	1.570 s

Observations

For 1 process, only rank 0 was executed and it displayed the "Hello World" message. When multiple processes were used, rank 0 displayed the "Hello World" message and the remaining processes displayed the normal process message. All processes were executed on the same machine, `DESKTOP-CQL9TFS`.

The execution time increased when more MPI processes were launched in this experiment. This can occur because the program performs very little computation, while starting and managing additional MPI processes introduces communication and process-management overhead.

GitHub

The Exercise 1 solution was committed and pushed to the GitHub repository:

IT24610803

The solution file is:

HelloMPI_solution.c

Exercise 2 – Parallel Sum Using MPI

Objective

To parallelize the addition of numbers from 1 to 10,000,000 using multiple MPI processes and measure the execution time for different numbers of processes.

Program

The MPI program divides the range of numbers among multiple processes. Each process calculates the sum of its assigned portion. The partial sums are then combined using MPI communication to obtain the total sum.

Compilation

```
mpicxx exercise2.cpp -o exercise2
```

Execution

The program was executed using 1, 2, 4, and 8 MPI processes.

```
mpirun -np 1 ./exercise2
mpirun -np 2 ./exercise2
mpirun -np 4 ./exercise2
mpirun --oversubscribe -np 8 ./exercise2
```

Results

Number of Processes	Execution Time (s)	Total Sum
1	0.0205822	50000005000000
2	0.0106076	50000005000000
4	0.0059839	50000005000000
8	0.00442634	50000005000000

Speedup

Speedup is calculated using:

Speedup = Execution Time with 1 Process / Execution Time with P Processes

Number of Processes	Execution Time (s)	Speedup
1	0.0205822	1.000
2	0.0106076	1.940
4	0.0059839	3.440
8	0.00442634	4.652

Observation

The execution time decreased as the number of MPI processes increased. The total sum remained the same for all process counts, confirming that the parallel program produced the expected result.

The measured speedup increased from 1.000 with one process to approximately 4.652 with eight processes. The speedup is not perfectly linear because of MPI communication and process-management overhead.

GitHub

The Exercise 2 solution was committed and pushed to the GitHub repository:

IT24610803

The solution file is:

exercise2.cpp

Exercise 3 – Estimating Pi Using MPI

Objective

To estimate the value of π using the Monte Carlo method and parallelize the calculation using MPI.

Method

The program generates random points within a square. Points that fall inside the quarter circle are counted. The value of π is estimated using the ratio between the points inside the circle and the total number of points.

The calculation is divided among multiple MPI processes, and the results from the individual processes are combined to obtain the final estimate of π .

Execution

The program was executed using 1, 2, 4, and 8 MPI processes.

```
mpirun -np 1 ./exercise3
mpirun -np 2 ./exercise3
mpirun -np 4 ./exercise3
mpirun --oversubscribe -np 8 ./exercise3
```

Results

Number of Processes	Total Points	Estimated Pi	Execution Time (s)
---------------------	--------------	--------------	--------------------

1	10,000,000	3.14064	0.131362
2	10,000,000	3.14166	0.0631034

Number of Processes	Total Points	Estimated Pi	Execution Time (s)
4	10,000,000	3.14123	0.0402513
8	10,000,000	3.14152	0.0323579

Observation

The estimated value of π was close to the mathematical value of π (approximately 3.14159) for all process counts. The small differences are expected because the Monte Carlo method uses randomly generated points.

The execution time decreased as the number of MPI processes increased. The lowest measured execution time was obtained using 8 processes.

The results demonstrate that MPI can be used to distribute the computational workload

Exercise 4 – Performance Analysis

For both Exercise 1 and Exercise 2, the execution time and speedup were analyzed using different numbers of MPI processes.

Exercise 1 – Time vs Number of Processors

The following graph shows the execution time of Exercise 1 for 1, 2, 4, and 8 MPI processes.

[Paste the Exercise 1 – Time vs Number of Processors graph here]

Exercise 1 – Speedup

The speedup was calculated using:

$$\text{Speedup} = T_1 / T_p$$

where T_1 is the execution time using one process and T_p is the execution time using P processes.

[Paste the Exercise 1 – Speedup graph here]

Exercise 2 – Time vs Number of Processors

The following graph shows the execution time of Exercise 2 for 1, 2, 4, and 8 MPI processes.

[Paste the Exercise 2 – Time vs Number of Processors graph here]

Exercise 2 – Speedup

The speedup for Exercise 2 was calculated using:

$$\text{Speedup} = T_1 / T_p$$

[Paste the Exercise 2 – Speedup graph here]

Analysis

The execution time generally decreased as the number of MPI processes increased, particula